

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LENGUAJES FORMALES Y AUTÓMATAS

PROYECTO FINAL

ALUMNO: Ortega Novoa Octavio

GRUPO: 7

## OBJETIVO:

Realizar el analizador léxico en flex y el analizador sintáctico en yacc de las sentencias de las sentencias de control del lenguaje C.

## DESARROLLO:

Cada equipo analizará el lenguaje de las sentencias de control de forma generalizada. De ello realizará lo siguiente:

Ejemplo:

Para la sentencia de control for la sintaxis es:

```
for (inicialización; condición; incremento/decremento) {  
    // Cuerpo del bucle  
}
```

Las expresiones regulares de cada uno de las cadenas

|          |                                        |
|----------|----------------------------------------|
| variable | $\{A-Za-z\}+(\_ \{A-Za-z\} \{0-9\}^*)$ |
| número   | $\{0-9\}^+\backslash\.\{0-9\}^+$       |
| for      | for                                    |
| ...      | ...                                    |

La gramática de cada una de las sentencias

```
G_for(P, T, NT, S)  
P:{  
    INICIO → for ( INTERIOR ) { }  
    INTERIOR → PRIMERO ; SEGUNDO ; TERCERO  
    PRIMERO → variable asignación número  
    SEGUNDO → variable comparación número  
    TERCERO → variable OPERACION  
    OPERACION → -- | ++  
}  
NT={INICIO, INTERIOR, PRIMERO, SEGUNDO, TERCERO, OPERACION INTERIOR}  
T={for, (, ), {, }, ; identificador, asignación, numero, comparación, --, ++}  
S={INICIO}
```

Obtener la tabla de clases.

Ejemplo:

Para el lenguaje C la tabla de clases:

| Clases |                     |
|--------|---------------------|
| 0      | Palabras Reservadas |
| 1      | Operadores          |
| 2      | Símbolos Especiales |
| 3      | Números             |
| 4      | Variables           |
| 5      | Cadenas             |
| ...    | ...                 |

Obtener la tabla para cada clase

Ejemplo:

La tabla de la clase Palabra reservadas:

| Palabras Reservadas |       |
|---------------------|-------|
| 0                   | for   |
| 1                   | if    |
| 2                   | else  |
| 3                   | while |
| ...                 | ...   |

La tabla de la clase operadores:

| Operadores |     |
|------------|-----|
| 0          | +   |
| 1          | -   |
| 2          | *   |
| 3          | /   |
| ...        | ... |

La tabla de la clase símbolos especiales:

| Símbolos Especiales |     |
|---------------------|-----|
| 0                   | (   |
| 1                   | )   |
| 2                   | {   |
| 3                   | }   |
| ...                 | ... |

Para la tabla de variables se tendrá un tratamiento especial el cual consiste en; cada nueva variable reconocida se agregará a la tabla de variables no

| Variables |      |
|-----------|------|
| 0         | m    |
| 1         | i    |
| 2         | var1 |
| 3         | var2 |
| ...       | ...  |

Para la tabla de cadenas se agrega cada cadena a la tabla, las cadenas deben de estar delimitadas por " o ' y solo puede aceptar letras, números y caracteres [-, \_, /, (, ), i, !, &, %, así como el punto y la coma].

La tabla de variables puede ser:

| Cadenas |                               |
|---------|-------------------------------|
| 0       | "Hola mundo"                  |
| 1       | 'Ingrese un numero del 0 - 9' |
| 2       | 'Esto es una cadena'          |
| 3       | "Bienvenido"                  |
| ...     | ...                           |

Importante: Como delimitador de un componente léxico será uno o varios espacios, tabuladores o saltos de línea.

Del análisis elaborado por equipo deberá realizar lo siguiente:

Mandar la impresión de los tokens a un archivo, en la terminal solo deberá indicar si el análisis **léxico** es incorrecto y se enviará a una tabla de errores la cual se imprimirá en otro archivo junto con la tabla de variables y la tabla de cadenas, si es correcto no dirá nada pero enviará el token al analizador sintáctico, por último indicará si análisis sintáctico es correcto o incorrecto.

| Errores |                              |
|---------|------------------------------|
| 0       | For                          |
| 1       | Ingrese un numero del 0 al 9 |
| 2       | l_var                        |
| 3       | \$                           |
| ...     | ...                          |

La impresión de los tokens en el archivo se realizará de la siguiente manera:

|       |                             |       |
|-------|-----------------------------|-------|
| Token | Identificador               | Clase |
| token | (id_clase, id_tabla_clase ) | Clase |

### Flex

El archivo flex recibirá la entrada desde un archivo externo, reconocerá cada token y lo enviará a Yacc, analizará si el token es correcto o incorrecto y en caso de ser correcto realizará el análisis de las clases, solo imprimirá aquellos tokens incorrectos.

### Yacc

El archivo yacc será un único archivo que analizará la gramática de las sentencias de control e indicará si la gramática es correcta y hará el correcto tratamiento de errores.

- Se generó el siguiente conjunto de Expresiones Regulares y tokens en el analizador léxico:

```
digit [0-9]
letter [a-zA-Z]
character ['^']
comparison <|>|<=|>|=
emptySpace [ \t\n]

%%
"int"           { registerToken(yytext, "Palabras Reservadas"); return INT; }
"float"         { registerToken(yytext, "Palabras Reservadas"); return FLOAT; }
"char"          { registerToken(yytext, "Palabras Reservadas"); return CHAR; }
"for"           { registerToken(yytext, "Palabras Reservadas"); return FOR; }
"if"            { registerToken(yytext, "Palabras Reservadas"); return IF; }
"else"          { registerToken(yytext, "Palabras Reservadas"); return ELSE; }
"while"         { registerToken(yytext, "Palabras Reservadas"); return WHILE; }
{letter}+(_|{letter}|{digit})*
{digit}+
{digit}+\. {digit}+
{comparison}
{emptySpace}+
'{character}?'
"="
"+"
"_"
"*"
"/"
"++"
"--"
"("
")"
"{"
"}"
";"
.
%%
{ registerToken(yytext, "Cadenas"); return CHAR_LITERAL; }
{ registerToken(yytext, "Operadores"); return ASSIGNATION; }
{ registerToken(yytext, "Operadores"); return PLUS; }
{ registerToken(yytext, "Operadores"); return MINUS; }
{ registerToken(yytext, "Operadores"); return MULTIPLY; }
{ registerToken(yytext, "Operadores"); return DIVIDE; }
{ registerToken(yytext, "Operadores"); return INCREMENT; }
{ registerToken(yytext, "Operadores"); return DECREMENT; }
{ registerToken(yytext, "Simbolos Especiales"); return OPEN_PARENTHESIS; }
{ registerToken(yytext, "Simbolos Especiales"); return CLOSE_PARENTHESIS; }
{ registerToken(yytext, "Simbolos Especiales"); return OPEN_BRACE; }
{ registerToken(yytext, "Simbolos Especiales"); return CLOSE_BRACE; }
{ registerToken(yytext, "Simbolos Especiales"); return SEMICOLON; }
{ registerToken(yytext, "Errores"); printf("Error lexico\n"); }
```

- Se utiliza la siguiente función para registrar cada token y almacenarlo en un arreglo, este arreglo contendrá estructuras almacenadas en memoria heap donde se almacenará cada token con su respectiva clase:

```
void registerToken(char *value, char *type)
{
    if(tokenCount >= MAX_TOKENS)
    {
        printf("Numero maximo de Tokens alcanzado\n");
        exit(1);
    }

    TokenData *newToken = malloc(sizeof(TokenData));
    newToken -> token = strdup(value);
    newToken -> type = strdup(type);
    allTokens[tokenCount] = newToken;
    tokenCount++;
}
```

- Por lo tanto, la sección de definiciones queda de la siguiente manera:

```
%{
#include <stdio.h>
#include <string.h>
#include "y.tab.h"

#define MAX_TOKENS 1000

typedef struct
{
    char *token;
    char *type;
} TokenData;

TokenData **allTokens;
int tokenCount = 0;

void registerToken(char *value, char *type);
void writeTokenTypes();
void writeTokenTable();
void writeTypeInTable(FILE *file, char *type);
%}
```

- Para escribir cada token con su clase en un archivo externo, se utiliza la siguiente función:

```
void writeTokenTypes()
{
    FILE *file = fopen("tokens.txt", "w");

    if(file == NULL)
    {
        printf("Error al abrir el archivo de escritura para tokens\n");
        exit(1);
    }

    fprintf(file, "\t\t\tToken\t\t\tClase\n\n");

    for(int i = 0; i < tokenCount; i++)
    {
        char *currentToken = allTokens[i] -> token;
        char *currentType = allTokens[i] -> type;

        fprintf(file, "%d\t\t\t%s\t\t\t%s\n", i, currentToken, currentType);
    }

    fclose(file);
}
```

- Para escribir la tabla de variables, cadenas y errores se utilizan las siguientes funciones:

```
void writeTokenTable()
{
    FILE *file = fopen("table.txt", "w");

    if(file == NULL)
    {
        printf("Error al abrir el archivo de escritura para tokens\n");
        exit(1);
    }

    writeTypeInTable(file, "Variables");
    writeTypeInTable(file, "Cadenas");
    writeTypeInTable(file, "Errores");

    fclose(file);
}

void writeTypeInTable(FILE *file, char *type)
{
    fprintf(file, "Tabla %s:\n", type);

    int count = 0;

    for(int i = 0; i < tokenCount; i++)
    {
        char *currentType = allTokens[i] -> type;
        int success = strcmp(currentType, type);

        if(success == 0)
        {
            char *currentToken = allTokens[i] -> token;
            fprintf(file, "%d\t\t\t\t%s\n", count, currentToken);
            count++;
        }
    }

    fprintf(file, "\n");
}
```



- Para el correcto control de todo el programa se utilizan las siguientes funciones:

```
void clearMemoryAllocation()
{
    for(int i = 0; i < tokenCount; i++)
    {
        free(allTokens[i]);
    }

    free(allTokens);
}

void readInputFile(int argc, char *argv[])
{
    if(argc > 1)
    {
        yyin = fopen(argv[1], "rt");
    }
    else
    {
        printf("Error al abrir el archivo de input\n");
        exit(1);
    }
}

int yywrap()
{
    return 1;
}
```

- Finalmente creamos la función main que leerá el analizador léxico y en analizador sintáctico utilizando yylex() y yywrap(), además de leer el archivo de entrada:

```
int main(int argc, char *argv[])
{
    readInputFile(argc, argv);

    allTokens = (TokenData **)malloc(MAX_TOKENS * sizeof(TokenData *));

    yylex();

    if(yyparse() == 0)
    {
        printf("Análisis sintactico correcto\n");
    }

    writeTokenTypes();
    writeTokenTable();
    clearMemoryAllocation();

    fclose(yyin);

    return 0;
}
```

- El analizador sintáctico reconocerá los siguientes tokens:

```
%{  
#include <stdio.h>  
void yyerror(char *msg);  
int yylex();  
%}  
  
%token INT  
%token FLOAT  
%token CHAR  
%token FOR  
%token IF  
%token ELSE  
%token WHILE  
%token IDENTIFIER  
%token INT_NUMBER  
%token FLOAT_NUMBER  
%token CHAR_LITERAL  
%token COMPARISON  
%token ASSIGNATION  
%token PLUS  
%token MINUS  
%token MULTIPLY  
%token DIVIDE  
%token INCREMENT  
%token DECREMENT  
%token OPEN_PARENTHESIS  
%token CLOSE_PARENTHESIS  
%token OPEN_BRACE  
%token CLOSE_BRACE  
%token SEMICOLON  
%start program
```

- Regla que identifica cada posible expresión (identificadores, números, caracteres e incrementos):

```
expression:  
    IDENTIFIER  
    | INT_NUMBER  
    | FLOAT_NUMBER  
    | CHAR_LITERAL  
    | IDENTIFIER INCREMENT  
    | IDENTIFIER DECREMENT;  
%%
```

- Regla que identifica cada tipo de dato (int, char, float):

```
type:
  INT
  | FLOAT
  | CHAR
```

- Regla que identifica cualquier operación con recursividad (suma, resta, multiplicación y división):

```
operation:
  expression
  | expression PLUS operation
  | expression MINUS operation
  | expression MULTIPLY operation
  | expression DIVIDE operation;
```

- Regla que identifica cualquier condición:

```
condition:
  operation
  | operation COMPARISON operation;
```

- Regla que identifica la expresión while:

```
while_statement:
  WHILE OPEN_PARENTHESIS condition CLOSE_PARENTHESIS OPEN_BRACE program CLOSE_BRACE;
```

- Regla que identifica la expresión if seguida de un else:

```
if_statement:
  IF OPEN_PARENTHESIS condition CLOSE_PARENTHESIS OPEN_BRACE program CLOSE_BRACE
  | IF OPEN_PARENTHESIS condition CLOSE_PARENTHESIS OPEN_BRACE program CLOSE_BRACE ELSE OPEN_BRACE program CLOSE_BRACE;
```

- Regla que identifica la expresión if seguida de un else:

```
for_statement:
  FOR OPEN_PARENTHESIS declaration SEMICOLON condition SEMICOLON operation CLOSE_PARENTHESIS OPEN_BRACE program CLOSE_BRACE;
```

- Regla que identifica la declaración de cualquier variable de cualquier tipo, incluyendo su respectiva inicialización y ajuste:

```
declaration:
    type IDENTIFIER
    | type IDENTIFIER ASSIGNATION operation
    | IDENTIFIER ASSIGNATION operation;
```

- Regla que generaliza cualquier otra regla de la gramática:

```
statement:
    declaration SEMICOLON
    | for_statement
    | if_statement
    | while_statement
    | operation SEMICOLON;
```

- Regla para todo el programa, la cual utiliza recursividad y generaliza cualquier regla de la gramática, esta es referenciada dentro de los corchetes de las expresiones anteriormente mencionadas para poder realizar cualquier acción:

```
program:
    | statement program;
```

- Finalmente se manejan los errores sintácticos de la siguiente manera:

```
void yyerror(char *msg)
{
    printf("Error sintactico\n");
}
```

- Probamos con el siguiente archivo de texto, donde colocaremos pruebas para cada regla:

```
int a;  
int b = 10;  
float c = 10.00 / 89 * a;  
  
a = 100;  
  
for(int i = 0; i <= c; i++)  
{  
    for(b = 10; b == 123; i--)  
    {  
        for(int j = i; j >= 1000; 100 * 100)  
        {  
            if(j > c++)  
            {  
                char d = 'd';  
                char e = 'e';  
                char f = '$';  
            }  
            else  
            {  
                f = '9';  
            }  
        }  
    }  
}  
  
while(999 * 78 / 1234 <= 123 + 90)  
{  
    while(10)  
    {  
        if(123)  
        {  
            a = 1000;  
        }  
    }  
}
```

- Obtenemos el siguiente mensaje en la terminal después de compilar:

```
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> .\output file.txt  
Análisis sintáctico correcto  
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> 
```

- Obtenemos la siguiente lista de tokens con sus respectivas clases:

|    | Token | Clase               |    | Token | Clase               |
|----|-------|---------------------|----|-------|---------------------|
|    |       |                     | 31 | ;     | Simbolos Especiales |
|    |       |                     | 32 | i     | Variables           |
| 0  | int   | Palabras Reservadas | 33 | ++    | Operadores          |
| 1  | a     | Variables           | 34 | )     | Simbolos Especiales |
| 2  | ;     | Simbolos Especiales | 35 | {     | Simbolos Especiales |
| 3  | int   | Palabras Reservadas | 36 | for   | Palabras Reservadas |
| 4  | b     | Variables           | 37 | (     | Simbolos Especiales |
| 5  | =     | Operadores          | 38 | b     | Variables           |
| 6  | 10    | Numeros             | 39 | =     | Operadores          |
| 7  | ;     | Simbolos Especiales | 40 | 10    | Numeros             |
| 8  | float | Palabras Reservadas | 41 | ;     | Simbolos Especiales |
| 9  | c     | Variables           | 42 | b     | Variables           |
| 10 | =     | Operadores          | 43 | ==    | Operadores          |
| 11 | 10.00 | Numeros             | 44 | 123   | Numeros             |
| 12 | /     | Operadores          | 45 | ;     | Simbolos Especiales |
| 13 | 89    | Numeros             | 46 | i     | Variables           |
| 14 | *     | Operadores          | 47 | --    | Operadores          |
| 15 | a     | Variables           | 48 | )     | Simbolos Especiales |
| 16 | ;     | Simbolos Especiales | 49 | {     | Simbolos Especiales |
| 17 | a     | Variables           | 50 | for   | Palabras Reservadas |
| 18 | =     | Operadores          | 51 | (     | Simbolos Especiales |
| 19 | 100   | Numeros             | 52 | int   | Palabras Reservadas |
| 20 | ;     | Simbolos Especiales | 53 | j     | Variables           |
| 21 | for   | Palabras Reservadas | 54 | =     | Operadores          |
| 22 | (     | Simbolos Especiales | 55 | i     | Variables           |
| 23 | int   | Palabras Reservadas | 56 | ;     | Simbolos Especiales |
| 24 | i     | Variables           | 57 | j     | Variables           |
| 25 | =     | Operadores          | 58 | >=    | Operadores          |
| 26 | 0     | Numeros             | 59 | 1000  | Numeros             |
| 27 | ;     | Simbolos Especiales | 60 | ;     | Simbolos Especiales |
| 28 | i     | Variables           | 61 | 100   | Numeros             |
| 29 | <=    | Operadores          | 62 | *     | Operadores          |
| 30 | c     | Variables           | 63 | 100   | Numeros             |
| 31 | ;     | Simbolos Especiales | 64 | )     | Simbolos Especiales |
| 32 | i     | Variables           | 65 | {     | Simbolos Especiales |
| 33 | ++    | Operadores          | 66 | if    | Palabras Reservadas |
| 34 | )     | Simbolos Especiales | 67 | (     | Simbolos Especiales |
| 35 | {     | Simbolos Especiales | 68 | j     | Variables           |

|     |       |                     |     |       |                     |
|-----|-------|---------------------|-----|-------|---------------------|
| 68  | j     | Variables           | 101 | (     | Simbolos Especiales |
| 69  | >     | Operadores          | 102 | 999   | Numeros             |
| 70  | c     | Variables           | 103 | *     | Operadores          |
| 71  | ++    | Operadores          | 104 | 78    | Numeros             |
| 72  | )     | Simbolos Especiales | 105 | /     | Operadores          |
| 73  | {     | Simbolos Especiales | 106 | 1234  | Numeros             |
| 74  | char  | Palabras Reservadas | 107 | <=    | Operadores          |
| 75  | d     | Variables           | 108 | 123   | Numeros             |
| 76  | =     | Operadores          | 109 | +     | Operadores          |
| 77  | 'd'   | Cadenas             | 110 | 90    | Numeros             |
| 78  | ;     | Simbolos Especiales | 111 | )     | Simbolos Especiales |
| 79  | char  | Palabras Reservadas | 112 | {     | Simbolos Especiales |
| 80  | e     | Variables           | 113 | while | Palabras Reservadas |
| 81  | =     | Operadores          | 114 | (     | Simbolos Especiales |
| 82  | 'e'   | Cadenas             | 115 | 10    | Numeros             |
| 83  | ;     | Simbolos Especiales | 116 | )     | Simbolos Especiales |
| 84  | char  | Palabras Reservadas | 117 | {     | Simbolos Especiales |
| 85  | f     | Variables           | 118 | if    | Palabras Reservadas |
| 86  | =     | Operadores          | 119 | (     | Simbolos Especiales |
| 87  | '\$'  | Cadenas             | 120 | 123   | Numeros             |
| 88  | ;     | Simbolos Especiales | 121 | )     | Simbolos Especiales |
| 89  | }     | Simbolos Especiales | 122 | {     | Simbolos Especiales |
| 90  | else  | Palabras Reservadas | 123 | a     | Variables           |
| 91  | {     | Simbolos Especiales | 124 | =     | Operadores          |
| 92  | f     | Variables           | 125 | 1000  | Numeros             |
| 93  | =     | Operadores          | 126 | ;     | Simbolos Especiales |
| 94  | 'g'   | Cadenas             | 127 | }     | Simbolos Especiales |
| 95  | ;     | Simbolos Especiales | 128 | }     | Simbolos Especiales |
| 96  | }     | Simbolos Especiales | 129 | }     | Simbolos Especiales |
| 97  | }     | Simbolos Especiales |     |       |                     |
| 98  | }     | Simbolos Especiales |     |       |                     |
| 99  | }     | Simbolos Especiales |     |       |                     |
| 100 | while | Palabras Reservadas |     |       |                     |
| 101 | (     | Simbolos Especiales |     |       |                     |
| 102 | 999   | Numeros             |     |       |                     |
| 103 | *     | Operadores          |     |       |                     |
| 104 | 78    | Numeros             |     |       |                     |
| 105 | /     | Operadores          |     |       |                     |



- Finalmente obtenemos la tabla de variables, cadenas y errores, donde observamos que no se registra ningún error:

|                  |      |
|------------------|------|
| Tabla Variables: |      |
| 0                | a    |
| 1                | b    |
| 2                | c    |
| 3                | a    |
| 4                | a    |
| 5                | i    |
| 6                | i    |
| 7                | c    |
| 8                | i    |
| 9                | b    |
| 10               | b    |
| 11               | i    |
| 12               | j    |
| 13               | i    |
| 14               | j    |
| 15               | j    |
| 16               | c    |
| 17               | d    |
| 18               | e    |
| 19               | f    |
| 20               | f    |
| 21               | a    |
| Tabla Cadenas:   |      |
| 0                | 'd'  |
| 1                | 'e'  |
| 2                | '\$' |
| 3                | '9'  |
| Tabla Errores:   |      |

- Después agregamos expresiones no definidas en el analizador léxico para obtener errores:

```
int a;
int b = 10;
float c = 10.00 / 89 * a;

a = 100;

for(int i = 0; i <= c; i++)
{
    for(b = 10; b == 123; i--)
    {
        __%@ = 123;
    }
}
```

- Obtenemos los siguientes mensajes cuando compilamos:

```
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> gcc lex.yy.c y.tab.c -o output
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> .\output file.txt
Error lexico
Error lexico
Error lexico
Error lexico
Error sintactico
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> []
```

- Finalmente obtenemos la tabla de variables, cadenas y errores, observamos que obtenemos los errores correctamente:

```
Tabla Variables:
0      a
1      b
2      c
3      a
4      a
5      i
6      i
7      c
8      i
9      b
10     b
11     i

Tabla Cadenas:

Tabla Errores:
0      —
1      —
2      %
3      @
```

- Para finalizar la fase de pruebas generamos código lleno de errores sintácticos:

```
int a;
= 10;
float c = 10.00 / 89 * a;

a = 100

for(int; i <= c; int a)
{
    for(b = 10; b == 123; i--)
}
```

- Obtenemos el siguiente mensaje cuando compilamos:

```
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> .\output file.txt
Error sintactico
PS C:\Users\Choi Beomgyu\Desktop\Proyecto LFyA> █
```

## CONCLUSIONES:

- Las herramientas de análisis léxico y sintáctico Flex y Yacc, demostraron ser de gran utilidad en el campo de la construcción y análisis de gramáticas en la programación. Estas herramientas proporcionan una estructura robusta para el diseño de compiladores, intérpretes y analizadores para una amplia gama de lenguajes de programación.
- Logramos implementar todos los temas vistos en el curso, como gramáticas regulares, autómatas finitos no deterministas, gramáticas libres de contexto, limpieza, etc. Las implementaciones anteriores ayudaron a reforzar los conocimientos adquiridos durante el curso, además de crear un panorama distinto sobre la construcción de nuevos lenguajes de programación.